# Verification and Validation Report: Bridging Gaps: AI for Diagram Accessibility

Team 22, Reading4All
Nawaal Fatima
Dhruv Sardana
Fiza Sehar
Moly Mikhail
Casey Francine Bulaclac

April 7, 2026

# 1 Revision History

Table 1: Revision History

| Date | Name | Notes |
|---|---|---|
| March 9, 2026 | All | Created initial draft of VnV Report |
| April 4, 2026 | Dhruv | Added brief introductory text under sections before subsections, and updated report formatting based on feedback. |
| April 5, 2026 | Dhruv | Added short lead-in paragraphs before subsection groups in sections that previously had no introductory text, to improve document flow and address formatting feedback. |
| April 6, 2026 | Dhruv | Peer Review `#602` Applied |
| April 7, 2026 | Dhruv | Peer Review `#594` Applied |
| April 4, 2026 | Francine B | Address Peer Feedback `#597`, `#600`, `#605` |
| April 5, 2026 | Moly M | Address Peer Feedback `#603` |

# 2 Introduction

This document presents the Verification and Validation (VnV) Report for the system under development. The purpose of this report is to demonstrate that the implemented system satisfies the functional and non-functional requirements specified in the Software Requirements Specification (SRS). The scope of this report includes verification activities such as system testing, unit testing, and traceability analysis, as well as validation activities including usability evaluation and performance assessment. These activities ensure that the system behaves as expected and meets stakeholder needs.

The testing methodology used in this report follows the strategy outlined in the VnV Plan. Test cases are designed to evaluate requirements, and results are documented to provide evidence of requirement coverage. The remainder of this document is organized as follows. Section 2 provides general information about the testing approach. Subsequent sections describe system tests, unit tests, traceability matrices, and validation activities. Finally, the report concludes with a summary of verification and validation results.

# 3 Symbols, Abbreviations, and Acronyms

## 3.1 Acronyms

Table 2: Abbreviations and Acronyms Used in the Verification and Validation Plan

| Acronym | Definition |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| AODA | Accessibility for Ontarians with Disabilities Act |
| AR | Accessibility Requirement |
| CAD | Canadian Dollar |
| CI | Continuous Integration |
| CD | Continuous Development |
| CR | Cultural/Regulatory Requirement |
| DP | Development Plan |
| FR | Functional Requirement |
| GUI | Graphical User Interface |
| HA | Hazard Analysis |
| LFR | Look and Feel Requirement |
| LR | Legal Requirement |
| MG | Module Guide |
| MIS | Module Interface Specification |
| ML | Machine Learning |
| MS | Maintenance Support Requirement |
| NFR | Nonfunctional Requirement |
| NVDA | NonVisual Desktop Access (screen reader) |
| OER | Operational and Environmental Requirement |
| OCR | Optical Character Recognition |
| PII | Personally Identifiable Information |
| PR | Performance Requirement |
| R | Requirement |
| RL | Release Requirement |
| RFT | Reliability and Fault Tolerance Requirement |
| SCR | Safety-Critical Requirement |
| SRS | Software Requirements Specification |
| SR | Security Requirement |
| SSO | Single Sign-On |
| TLS | Transport Layer Security |
| UI | User Interface |
| UHR | Usability and Humanity Requirement |
| VnV | Verification and Validation |
| WCAG | Web Content Accessibility Guidelines |

## 3.2 Symbolic Constants

| Name | Value |
|---|---|
| T_ALT_GEN_SMALL | 3 seconds(s) |
| T_ALT_GEN_LARGE | 8 s |
| T_UI_RESP | 300 miliseconds (ms) |
| R_SUFFICIENCY | 85% |
| R_LENGTH | 90% |
| R_USABILITY_MEDIAN | 3 (rating) |
| R_USABILITY_MIN | 2 (rating) |
| T_ERROR_HANDLE | 2 s |
| T_RECOVERY | 5 s |
| CAP_CONCURRENT | 2 requests |
| CAP_STORAGE | 500 images/day |
| MAINT_TIME | 2 person-days/quarter |
| COMPAT_VERSIONS | 2 releases |
| IMG_SIZE_BIG | 10 MEGABYTES (MB) |
| IMG_SIZE_SMALL | 2 MEGABYTES |
| TLS_VERSION | 1.2 |
| FILE_DELETE_TIME | 60 s |
| FILE_TYPES | .png, .jpg, .jpeg, .svg, .webp |
| NETWORK_SOURCE_POLICY | McMaster SSO tokens or IP ranges only |
| TEAM_SIZE | 5 students |
| HOURS_RESEARCH | 40 hours |
| HOURS_BACKEND | 120 hours |
| HOURS_FRONTEND | 80 hours |
| HOURS_TESTING | 60 hours |
| HOURS_DOCS | 30 hours |
| HOURS_TOTAL | 330 hours |
| HOURS_PROJECT | 1,320 person-hours |
| COST_PER_HOUR | $20/hour |
| COST_TOTAL | $26,400 CAD |
| COST_ACTUAL | $0 CAD |
| COST_INCENTIVE_MIN | $100 CAD |
| COST_INCENTIVE_MAX | $150 CAD |
| MAX_ZOOM_PERCENTAGE | 200% |
| MIN_CONTRAST_RATIO | 4.5:1 |

*Table continues on next page*

| | |
|---|---|
| MAX_UPLOAD_STEPS | 5 steps |
| MAX_MINUTES | 5 minutes |
| USERS_SUCCESS_PERCENT | 80% |
| MAX_ERROR_RECOVER | 2 seconds |
| LEARNING_PERCENT | 90% |
| MAX_LEARNING_MINUTES | 5 minutes |
| MIN_COMPENSATION_DOLLARS | $100 CAD |
| MAX_COMPENSATION_DOLLARS | $150 CAD |
| LATEST_RELEASES_NUM | 3 |
| MOST_COMMON_SR | 3 |
| SFWR_RELEASES | 2 |

# Contents


**1 Revision History** **i**

**2 Introduction** **ii**

**3 Symbols, Abbreviations, and Acronyms** **iv**
3.1 Acronyms . . . . . iv
3.2 Symbolic Constants . . . . . v

**4 Functional Requirements Evaluation** **1**

**5 Nonfunctional Requirements Evaluation** **4**
5.1 Internal Testing Results . . . . . 4
5.2 System Tests . . . . . 5

**6 Comparison to Existing Implementation** **13**

**7 Unit Testing** **13**
7.1 ImageValidation Module . . . . . 13
7.2 BackendController Module . . . . . 14
7.3 AuthenticationService Module . . . . . 18
7.4 MainScreen Module . . . . . 21
7.5 ShowHistory Module . . . . . 23
7.6 LoginScreen Module . . . . . 24

**8 Manual Testing** **26**
8.1 UserInterfaceInteractions Module . . . . . 26
8.2 UserInterfaceAccessibility Module . . . . . 29
8.3 DataPreProcess Module . . . . . 32
8.4 ModelOutput Module . . . . . 35
8.5 AIModelTraining Module . . . . . 36
8.6 CaptionGeneration Module . . . . . 36
8.7 InternalMetricCompliance Module . . . . . 37

**9 Changes Due to Testing** **38**
9.1 Fixing Session Timeout Handling . . . . . 38
9.2 Narrowing the Scope . . . . . 38
9.3 Shortening Alt-Text Generation Time . . . . . 38

9.4 Clearer User Navigation . . . . . 38
9.5 Future Work . . . . . 39

**10 Automated Testing** **39**
10.1 Accessibility Checker - Axe . . . . . 39
10.2 CI/CD . . . . . 40
10.3 Mutation Testing . . . . . 41

**11 Trace to Requirements** **41**

**12 Trace to Modules** **42**

**13 Code Coverage Metrics** **43**

# List of Tables

1 Revision History . . . . . i
2 Abbreviations and Acronyms Used in the Verification and Validation Plan . . . . . iv
4 System Tests Functional Requirements Evaluation . . . . . 1
5 Internal Evaluation Scores per Evaluator . . . . . 4
6 System Tests Non-Functional Requirements Evaluation . . . . 5
7 ImageValidation Unit Testing Evaluation . . . . . 13
8 BackendController Module Unit Testing Evaluation . . . . . 15
9 AuthenticationService Module Unit Testing Evaluation . . . . 19
10 MainScreen Unit Testing Evaluation . . . . . 21
11 ShowHistory Unit Testing Evaluation . . . . . 23
12 LoginScreen Unit Testing Evaluation . . . . . 24
13 UserInterfaceInterfactions Manual Testing Evaluation . . . . . 29
14 UserInterfaceAccessibility Manual Testing Evaluation . . . . . 32
15 DataPreprocess Manual Testing Evaluation . . . . . 34
16 Manual Testing Evaluation for ModelOutput Module . . . . . 36
17 Manual Testing Evaluation for AIModelTraining Module . . . 36
18 Manual Testing Evaluation for CaptionGeneration Module . . 37
19 Manual Testing Evaluation for InternalMetricCompliance Module . . . . . 37
20 Traceability Between Tests and Requirements . . . . . 42
21 Traceability Between Tests and Modules . . . . . 43

# List of Figures

1 Automated Report for "Upload Page" in the Reading4All System . . . . . 40
2 Coverage Report of the React Frontend . . . . . 44
3 Coverage Report of the Backend . . . . . 45

This document summarizes the verification and validation activities completed for the Reading4All system. Table 1 records the revision history, and Table 2 lists the abbreviations used throughout the report.

# 4 Functional Requirements Evaluation

The following section outlines the evaluation of the system tests created for the functional requirements. Table 4 summarizes the functional system test results.

Table 4: System Tests Functional Requirements Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| FR-ST1 | User uploads image files in different formats to the system (i.e: png, jpeg, pdf, gif) | The system accepts .jpeg and .png files for processing. Files with unsupported formats (.gif, .pdf) are rejected and an error message is displayed. | The system successfully accepted and processed the .jpeg and .png files. The .gif and .pdf files were rejected, and an error message was displayed. | Pass |
| FR-ST2 | User uploads test diagrams (diagram1.png, diagram2.jpeg) to the system and generate alternative text descriptions. Evaluate the generated alt text using the evaluation metrics defined in Appendix 6.1. | The system generates alternative text descriptions for each uploaded image. The descriptions meet the evaluation metrics in Appendix 6.1. | Alt text generated but did not meet the required evaluation metric thresholds. Specifically, the Learning Impact metric scored < 2. | Fail |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| FR-ST3 | Generate alternative text for an uploaded image, then use screen readers (NVDA, JAWS, and VoiceOver) to read the generated alternative text aloud. | The generated alternative text is fully read aloud by common screen readers (NVDA, JAWS, VoiceOver) without truncation, misreading, or formatting errors. | The screen reader VoiceOver successfully read the generated alternative text fully and clearly. | Pass |
| FR-ST4 | Edit the generated alternative text by adding words, deleting sentences, and modifying phrasing, then save the changes. | The system accurately reflects all user edits and stores the updated alternative text without data loss or formatting errors. | The user's edits (added, deleted, and modified text) were correctly reflected and saved. The updated alternative text was stored without data loss or formatting issues. | Pass |
| FR-ST5 | Upload more than `SESSION_HISTORY_MAX` images sequentially in the same session, then navigate or refresh the session interface to view the upload history. | Only the `SESSION_HISTORY_MAX` most recent uploads and alt text are visible. | Recent uploads up to the limit were displayed correctly. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| FR-ST6 | Attempt to log in using both valid McMaster credentials (McMaster email and password) and invalid credentials (non-McMaster email or incorrect password). | Users with valid McMaster University credentials are granted access to the system. Login attempts with invalid credentials are rejected and an error message is displayed. | Login with valid McMaster credentials was successful and access to the system was granted. Login attempts using invalid credentials were rejected and an error message was displayed. | Pass |
| FR-ST7 | Log into the system and remain inactive for longer than `SESSION_TIMEOUT`, then attempt to access the system again. | After `SESSION_TIMEOUT` of inactivity, the session expires automatically. The user is redirected or prompted to log in again before accessing the Reading4All system. | Session expired automatically after `SESSION_TIMEOUT` of inactivity. The user was redirected to the login page and required to authenticate again. | Pass |

**Analysis of Previous Failed Tests:**

- **FR-ST2**: These failures were primarily due to limitations in the AI model's training data and tuning. The model generated descriptions that were syntactically correct but lacked sufficient detail and educational value, leading to low scores in sufficiency and learning impact. This issue was not related to code correctness but rather to model performance and requirement expectations. Improvements such as additional training data and hypertuning are required to address this.
- **FR-ST7**: The session timeout functionality initially did not trigger as expected, indicating an issue in session lifecycle handling. This failure was

likely due to incomplete implementation or missing backend logic for enforcing session expiration rather than unclear requirements. After correcting the session management logic, the timeout functionality was verified to work as expected in subsequent testing.

# 5 Nonfunctional Requirements Evaluation

The following section evaluates how well the system meets the non-functional requirements outlined in the SRS. Testing was completed through internal evaluations of generated alt text using pre-defined rating metrics, as well as system tests aimed at determining the system's accessibility, usability and overall performance. These tests included both common usage scenarios and edge cases to ensure the system remains reliable under different conditions. The results are described in the following subsections.

## 5.1 Internal Testing Results

Five evaluators assessed the generated alt text using the metrics defined in Table 5. Each evaluator rated the output on Sufficiency of Description, Length Appropriateness, Accessibility/Usability, and Learning Impact. Table 5 summarizes the scores collected.

Table 5: Internal Evaluation Scores per Evaluator

| **Evaluator** | **Sufficiency (1–3)** | **Length (1–3)** | **Accessibility (0–3)** | **Learning (0–3)** | **Notes** |
|---|---|---|---|---|---|
| Person 1 | 2 | 3 | 3 | 1 | N/A |
| Person 2 | 2 | 3 | 3 | 2 | N/A |
| Person 3 | 2 | 3 | 3 | 1 | N/A |
| Person 4 | 2 | 2 | 3 | 1 | N/A |
| Person 5 | 2 | 2 | 3 | 1 | N/A |
| **Mean** | **2.0** | **2.6** | **3.0** | **1.2** | — |
| **Acceptable** | $\geq 3$ | $\geq 3$ | $\geq 2$ | $\geq 2$ | — |

The results indicate that Accessibility/Usability consistently met or exceeded the acceptable threshold across all evaluators, with a mean score of 3.0. Length Appropriateness averaged 2.6, falling slightly below the acceptable threshold of 3. Sufficiency of Description scored uniformly at 2.0, below the acceptable threshold of 3, suggesting that the generated alt text does not yet convey sufficient information to fully achieve the intended objective. Learning Impact averaged 1.2, also

below the acceptable threshold of 2, indicating that improvements are needed to better support user understanding in learning contexts.

## 5.2 System Tests

The following section outlines the evaluation of the system tests created for the nonfunctional requirements. Table 6 summarizes the nonfunctional system test results.

Table 6: System Tests Non-Functional Requirements Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST1 | User adjusts browser zoom to maximum and evaluates color contrast using accessibility tools | Text resizes correctly without overlap; contrast meets WCAG 2.1 AA accessibility requirements; images include alternative text | Tester confirms accessibility compliance using zoom tests, contrast checkers, and screen readers | Pass |
| NFR-ST2 | Tester visually inspects layout, typography, colors, and branding elements of the interface | Interface maintains consistent styling; McMaster branding is present and does not interfere with usability; design aligns with Norman's principles | Tester verifies UI against style guide, branding rules, and Norman's usability principles | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST3 | Users perform key tasks such as login, image upload, and alt-text generation | Tasks are completed efficiently; feedback is received within FEED-BACK_TIME; users can recall steps and correct errors easily | Tester evaluates usability through timed task completion, feedback monitoring, and observation of user error recovery | Pass |
| NFR-ST4 | User generates alt text and selects the copy or storage option | Alt text is successfully copied to the clipboard or stored as requested | Tester verifies each option by generating alt text and confirming correct execution of the selected storage method | Pass |
| NFR-ST5 | Screen reader users upload images and generate alt text using assistive technologies such as NVDA, JAWS, or VoiceOver | Users successfully generate and obtain alt text within the time limit and navigate the interface efficiently using screen readers | Tester conducts usability sessions with screen reader tools and measures task completion time and navigation success | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST6 | Images of different sizes were uploaded under typical system conditions to evaluate alt-text generation time | Alt text should be generated within T_ALT_GEN_SMALL and T_ALT_GEN_LARGE thresholds, while UI responsiveness remains within T_UI_RESP | Image validity was unit tested, while generation time was measured manually during testing; the UI responsiveness and model response time were observed and recorded | Fail |
| NFR-ST7 | Tester simulates long-running or stalled image processing to trigger a timeout condition | User is notified of the timeout with an option to retry; incomplete data is deleted and no technical details are exposed in messages | Tester verifies timeout notifications, retry option, proper data cleanup, and absence of sensitive technical information in error messages | Pass |
| NFR-ST8 | Conduct user evaluation of generated alt text using defined rating metrics (sufficiency, length, accessibility, learning impact) | Average ratings meeting required thresholds: R_SUFFICIENCY, R_LENGTH, and R_USABILITY _MEDIAN | Average ratings: sufficiency= 2.0, length= 3.0, learning impact= 1.33. Only length and accessibility met thresholds | Fail |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST9 | Uploads unsupported fil or corrupted files and simulate backend process failures in order to evaluate system error handling and recovery. | System displays a clear error message and recovers within T_RECOVERY seconds. | System displays to user a clear error message and recovers in 1.8 seconds, which is within the required threshold of T_RECOVERY | Pass |
| NFR-ST10 | Simulate multiple users logged into the system simultaneously, each uploading different images and requesting alt text. | System supports CAP_CONCURRENT users with response times less than CAPACITY_RTIME and sucessfullly handles datasets up to CAP_STORAGE. | System handles 3 concurrent users uploading images with response times within the required threshold and stored the uploaded data successfully. | Pass |
| NFR-ST11 | Replace the AI model with a different one and update any required configuration (ex, API endpoint) while keeping the rest of the system unchanged. Run tests to verify functionality. | System continues functioning correctly. The new AI model integrates with only minimal configuration changes and does not break existing components. | The new AI model was integrated successfully with only API endpoint changes, and all main functionalities work as expected. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST12 | Attempt logins from non-mcmaster accounts, upload unsupported files, verify HTTPS encryption and confirm system logs access is restricted. | Only McMaster users can sign-up and access the system, unsupported files are rejected; communication is encrypted and system logs are restricted | Login and sign up attempts from users outside the McMaster domain are rejected, unsupported files are rejected, and the application is access through HTTPS, confirming encrypted communication. Logs are currently only accessible by authorized individuals. | Pass |
| NFR-ST13 | Generate alt text for mutliple physics diagrams (ex, circuits, force diagrams and scenarios involving people) and manually review the text generated for bias, cultural references and tone. | Generated alt text is neutral, inclusive, contextually accurate and written in a professional tone without any cultural references. | The generated alt text remained neutral and focused on physics concepts and did not introduce any bias. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST14 | Generate alt text for multiple diagrams and review outputs and system documentation for compliance with accessibility standards. | Generated alt text meets accessibility standards, uploaded files are handling according to policy and documentation is available to stakeholders. | Generated alt text met accessibility standards, uploaded files were handles correctly; however, application instructions are not available to stakeholders yet. | Fail |
| NFR-ST15 | Test device and browser compatibility | System runs correctly on supported operating systems and browsers | All assertions pass | Pass |
| NFR-ST16 | Evaluate alt text accuracy metrics | Generated alt text meets sufficiency, length, and usability thresholds | All assertions pass | Pass |
| NFR-ST17 | Verify image privacy protection | Uploaded images are deleted after processing and not permanently stored | All assertions pass | Pass |
| NFR-ST18 | Check for biased or offensive alt text | Generated alt text contains no harmful, biased, or inappropriate content | All assertions pass | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST19 | Perform WCAG accessibility audit | Interface satisfies WCAG 2.1 accessibility guidelines | All assertions pass | Pass |
| NFR-ST20 | Test screen reader compatibility | Screen readers correctly read generated alt text | All assertions pass | Pass |
| NFR-ST21 | Test supported image formats | System successfully processes JPG, JPEG, and PNG images | All assertions pass | Pass |
| NFR-ST22 | Run the system website through Axe (third-party service) to check for WCAG 2.1 AA compliance. | There are no issues for all website pages | Some JSON results reported 'moderate' or 'best practices' issues in the summary. | Fail |
| NFR-ST23 | System website is hosted on Vercel (third-party service). | Website is in service and usable with minimal to no crashes. | Website is fully functional on the Vercel server. No crashing detected. | Pass |
| NFR-ST24 | System website is hosted on a testing environment. | Core functions (uploading an image, text generation, and user navigation) function as intended. | Core functionality remains intact on testing environment. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| NFR-ST25 | System website is deployed into staging environment. | System is fully ready for DEMO deliverable as per Capstone's March timeline. | The final prototype was finised as per the requirements and a DEMO was conducted successfully. | Pass |
| NFR-ST26 | When executing actions in the system, logs are available for API interactions and standard workflows | Logs are visible for any crash or API call. | Logging module was implemented and appropriate level of logs are shown. | Pass |

**Analysis of Previous Failed Tests:**

- **NFR-ST6:** The system did not meet the required performance threshold for alt-text generation time. This was caused by the computational cost of the AI model and lack of optimization rather than poor code quality. The issue was partially addressed by switching to a faster model, as described in Section 8.2.
- **NFR-ST8:** These failures were primarily due to limitations in the AI model's training data and tuning. The model generated descriptions that were syntactically correct but lacked sufficient detail and educational value, leading to low scores in sufficiency and learning impact. This issue was not related to code correctness but rather to model performance and requirement expectations. Improvements such as additional training data and hypertuning are required to address this.
- **NFR-ST14 and NFR-ST22:** These failures were due to missing UI elements (e.g., main landmark) and incomplete implementation of accessibility documentation and best practices. The issues stem from implementation gaps and evolving requirements rather than fundamental design flaws. Automated tools (e.g., Axe) helped identify these gaps for future correction.

- **NFR-ST25:** Failures in staging readiness were due to incomplete integration and unresolved issues from earlier failed tests. This indicates that the system was not fully stabilized before deployment rather than issues with individual modules.

# 6 Comparison to Existing Implementation

This section will not be appropriate for every project.

# 7 Unit Testing

The following are the evaluation of the unit tests per module. The module-level unit test results are summarized in the tables throughout this section, beginning with Table 7.

## 7.1 ImageValidation Module

Table 7 summarizes the unit testing results for the ImageValidation module.

Table 7: ImageValidation Unit Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| IMGVAL-UT1 | Validate request with missing image. | The module returns `"MISSING_IMAGE"` to indicate the missing image. | The module returned `"MISSING_IMAGE"`. | Pass |
| IMGVAL-UT2 | Validate request with invalid file type. | The module returns `"INVALID_FILE_TYPE"` to indicate that the uploaded file is not a valid image type. | The module returned `"INVALID_FILE_TYP` | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| IMGVAL-UT3 | Validate request with an image that exceeds the size limit. | The module returns `"FILE_SIZE_INVALID"` to indicate that the uploaded image exceeds the size limit. | The module returned `"FILE_SIZE_INVALI` | Pass |
| IMGVAL-UT4 | Validate request with a corrupted image file. | The module returns `"UNAUTHORIZED_ACCESS_OR_CORRUPTED"` to indicate that the uploaded image cannot be accessed. | The module returned `"UNAUTHORIZED_ACCESS_OR_CORRUPTED"`. | Pass |
| IMGVAL-UT5 | Validate request with a valid image file. | The module returns `"Success"` to indicate that the uploaded image meets system requirements and has been successfully uploaded. | The module returned `"Success"`. | Pass |

## 7.2 BackendController Module

Table 8 summarizes the unit testing results for the BackendController module.

Table 8: BackendController Module Unit Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| BCONT-UT1 | Send image validation request with valid image data through the backend controller. | The backend controller forwards the image to the Image Validation module and returns the validation results. | The backend controller correctly returns `Success` after calling the image validation module. | Pass |
| BCONT-UT2 | Send alt text generation request with image data and a valid session ID through the backend controller. | The backend controller forwards the request to the Alt Text Generation module and returns the generated alt text. | The backend controller correctly returns the generated alt text of `"Generated alt text"`. | Pass |
| BCONT-UT3 | Send alt text history retrieval request with a valid session ID through the backend controller. | The backend controller forwards the session ID to the Alt Text History module and returns the retrieved history. | The backend controller correctly returns the retrieved history of `["Alt text 1", "Alt text 2"]`. | Pass |
| BCONT-UT4 | Send alt text editing request through the backend controller. | The backend controller forwards the request to the Edit Alt Text module and returns the result. | The backend controller correctly returns the editing result of `"Success"`. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| EDIT-UT1 | Submit a valid edit request with valid entry ID, edited alt text and session ID. | The alt text is updated with the edited text and returns a success message. | The service correctly returns `"Success"` after calling the service to update the history data. | Pass |
| EDIT-UT2 | Submit a valid edit request but simulate a failed save to the database. | The service returns and error indicating the change cannot be saved. | The service correctly returns an error message of `"UNABLE_TO_SAVE"` and does not call the service to update the history data. | Pass |
| EDIT-UT3 | Submit an edit request with the `entry_id` field missing. | The service detects the missing field and returns an error indicating the missing field and does not attempt to update the alt text. | The service correctly returns an error message of `"INVALID_REQUEST"` and does not call the service to update the history data. | Pass |
| EDIT-UT4 | Submit an edit request with the `edited_alt_text` field missing. | The service detects the missing field and returns an error indicating the missing field and does not attempt to update the alt text. | The service correctly returns an error message of `"INVALID_REQUEST"` and does not call the service to update the history data. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| EDIT-UT5 | Submit an edit request but simulate an exception being thrown by the Supabase client. | The service catches the exception and returns an error indicating the failure and does not attempt to update the alt text. | The service correctly returns an error message of `"UNABLE_TO_SAVE"` after the exception is caught | Pass |
| GEN-UT1 | Submit alt text with extra spaces for post-processing. | The extra spaces are removed from the alt text and its returned. | The function correctly removes extra spaces from the alt text. | Pass |
| GEN-UT2 | Submit alt text with repeated words for post-processing. | The repeated words are removed from the alt text and it is returned. | The function correctly removes repeated words from the alt text. | Pass |
| GEN-UT3 | Submit alt text starting with a lowercase letter for post-processing. | The first letter is capitalized and it is returned. | The function correctly capitalizes the first letter of the alt text. | Pass |
| GEN-UT4 | Submit alt text with a missing period for post-processing. | A period is added to the end of the alt text and it is returned. | The function correctly adds a period to the end of the alt text. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| GEN-UT5 | Submit alt text with multiple sentences beginning with lowercase letters for post-processing. | Each sentence is capitalized and the text is returned. | The function correctly capitalizes each sentence in the alt text. | Pass |
| GEN-UT6 | Trigger the AI model for AI alt text generation and simulate no text is returned | The function returns no alt text and no update to history is completed. | The function correctly returns `(None, None)` | Pass |
| GEN-UT7 | Trigger the AI model for AI alt text generation and simulate alt text is returned | The function returns the generated alt text and updates the history. | The function correctly returns the generated alt text and updates the history. | Pass |

## 7.3 AuthenticationService Module

Table 9 summarizes the unit testing results for the AuthenticationService module.

Table 9: AuthenticationService Module Unit Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| AUTH-UT1 | Send a request with a valid email and password to the AuthenticationService. | The authentication service successfully creates a new user account and returns the user ID and access token. | The authentication service correctly returns the user ID and access token, i.e., {`"user_id": "test_user_id", "access_token": "test_access_toke"`} | Pass |
| AUTH-UT2 | Send request with invalid email format to the Authentication-Service | The authentication service raises a `ValueError` with the message "Email must be a @mcmaster.ca address". The supabase client is not called. | The authentication service correctly raises a `ValueError` with the message "Email must be a @mcmaster.ca address" and the supabase client is not called. | Pass |
| AUTH-UT3 | Send a request with a password shorter than 8 characters to the Authentication-Service. | The authentication service raises a `ValueError` with the message "Password must be at least 8 characters". The Supabase client is not called. | The authentication service correctly raises a `ValueError` with the message "Password must be at least 8 characters" and the Supabase client is not called. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| AUTH-UT4 | Send a request with empty email and password to the AuthenticationService. | The authentication service raises a `ValueError` with the message "Email must be a @mcmaster.ca address". The Supabase client is not called. | The authentication service correctly raises a `ValueError` with the message "Email must be a @mcmaster.ca address" and the Supabase client is not called. | Pass |
| AUTH-UT5 | Send a request with `STUDENT@mcmaster.ca` as the email to the AuthenticationService. | The authentication service correctly converts the email to lowercase before processing. | The authentication service correctly converts the email to lowercase before processing. The Supabase client is called with `student@mcmaster.ca` | Pass |
| AUTH-UT6 | Send a request with an existing email to the AuthenticationService. | The authentication service raises a `ValueError` with the message "Email is already in use". A verification email is not sent to the user | No error was raised and the user received a Verification email. | Fail |

**Analysis of Failed Tests:**

- **AUTH-UT6:** The failure in AUTH-UT6 occurred because the system al-

lowed account creation using an email that already exists, instead of raising an error as expected. This indicates that duplicate account prevention was not properly enforced at the application level. The issue is likely due to reliance on the external authentication service (Supabase), which may not restrict duplicate sign-up attempts by default, or the absence of an explicit validation check before account creation. This reflects an implementation gap rather than a requirement ambiguity or code quality issue. Future improvements should include enforcing unique user constraints and validating existing accounts prior to registration.

## 7.4 MainScreen Module

Table 10 summarizes the unit testing results for the MainScreen module.

Table 10: MainScreen Unit Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| MAIN-UT1 | Render HomeScreen page | Homepage title is displayed | Homepage title displayed successfully | Pass |
| MAIN-UT2 | Render HomeScreen subtitle | Homepage subtitle describing system purpose is visible | Subtitle displayed correctly | Pass |
| MAIN-UT3 | Display signup button on homepage | Signup button is visible for new users | Signup button rendered successfully | Pass |
| MAIN-UT4 | Display login button on homepage | Login button is visible for returning users | Login button rendered successfully | Pass |
| MAIN-UT5 | Render UploadScreen title | Upload page title is visible | Title displayed correctly | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| MAIN-UT6 | Render UploadScreen subtitle | Upload page subtitle guiding users is visible | Subtitle displayed correctly | Pass |
| MAIN-UT7 | Verify upload instructions accessibility | Upload instructions accessible through ARIA label | ARIA label detected successfully | Pass |
| MAIN-UT8 | Verify upload input accessibility label | Upload input field contains accessible label | Label detected successfully | Pass |
| MAIN-UT9 | Display Choose File button | Choose File button is visible for uploading images | Button displayed successfully | Pass |
| MAIN-UT10 | Upload valid image file | Image successfully uploaded and backend request triggered | Image uploaded and confirmation displayed | Pass |
| MAIN-UT11 | Upload image using drag-and-drop | Image uploaded using drag-and-drop functionality | Drag-and-drop upload completed successfully | Pass |
| MAIN-UT12 | Remove uploaded image | Uploaded image removed from interface | Image removed successfully | Pass |
| MAIN-UT13 | Generate alt text after image upload | Generated alt text displayed in output field | Alt text generated and displayed successfully | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| MAIN-UT14 | Upload new image after generation | Previous alt text resets when new image uploaded | Alt text reset confirmed | Pass |
| MAIN-UT15 | Copy generated alt text | Alt text copied to clipboard successfully | Clipboard copy function executed successfully | Pass |
| MAIN-UT16 | Keyboard accessibility for file upload | Pressing Enter opens file selection dialog | File input triggered successfully | Pass |
| MAIN-UT17 | Handle image upload error | Error message displayed when upload fails | Error message displayed correctly | Pass |
| MAIN-UT18 | Handle alt text generation failure | Error message displayed when alt text generation fails | Error message displayed correctly | Pass |

## 7.5 ShowHistory Module

Table 11 summarizes the unit testing results for the ShowHistory module.

Table 11: ShowHistory Unit Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| HIST-UT1 | Render ShowHistory page | History page title and navigation button are displayed | Title and back button displayed successfully | Pass |
| HIST-UT2 | Display history entries | Uploaded image history entries are listed on the page | History entries rendered successfully | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| HIST-UT3 | View history entry preview | Clicking a history entry displays the image preview and alt text | Preview image and alt text displayed correctly | Pass |
| HIST-UT4 | Navigate back to upload screen | Clicking the back button navigates to the upload page | Navigation executed successfully | Pass |

## 7.6 LoginScreen Module

Table 12 summarizes the unit testing results for the LoginScreen module.

Table 12: LoginScreen Unit Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| LOGIN-UT1 | Render login interface | Login page title and input fields are visible | Login interface rendered successfully | Pass |
| LOGIN-UT2 | Submit login with empty fields | System rejects login attempt and displays validation error | Validation error displayed successfully | Pass |
| LOGIN-UT3 | Enter email and password credentials | User can type and update values in email and password fields | Input fields accept user input correctly | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| LOGIN-UT4 | Login with valid credentials | System sends authentication request and redirects user to upload page | Authentication request successful and navigation executed | Pass |
| LOGIN-UT5 | Login with invalid credentials | System displays authentication error message | Error message displayed correctly | Pass |
| LOGIN-UT6 | Network failure during login | System displays network error message | Network error message displayed successfully | Pass |
| LOGIN-UT7 | Render signup form interface | Signup form fields and instructions are displayed | Signup interface rendered successfully | Pass |
| LOGIN-UT8 | Signup validation with missing email | System displays validation error when email is not provided | Validation error displayed successfully | Pass |
| LOGIN-UT9 | Signup validation for short password | System rejects passwords shorter than minimum length | Password validation message displayed | Pass |
| LOGIN-UT10 | Signup validation for mismatched passwords | System rejects mismatched password entries | Mismatch validation error displayed | Pass |
| LOGIN-UT11 | Send verification code | Verification code request sent to backend and OTP field enabled | Verification request executed successfully | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| LOGIN-UT12 | Signup attempt without OTP | System prevents signup and displays OTP required error | OTP validation error displayed | Pass |
| LOGIN-UT13 | Complete successful signup | System verifies OTP and completes account registration | Signup process completed successfully | Pass |
| LOGIN-UT14 | RequireAuth loading state | Protected content not rendered until authentication check completes | Content hidden during authentication check | Pass |
| LOGIN-UT15 | RequireAuth authenticated access | Authenticated users can access protected content | Protected content rendered successfully | Pass |
| LOGIN-UT16 | RequireAuth unauthorized access | Unauthenticated users are redirected to login page | Unauthorized access prevented and redirect executed | Pass |

# 8 Manual Testing

This section summarizes the manual tests carried out for the major system modules and presents the observed results.

## 8.1 UserInterfaceInteractions Module

The following are the manual tests for this module.

UII-MT 1. **Initial State:** User is logged in and the main screen is displayed.
**Input:** User selects and uploads an image file using the upload button.
**Test Steps:**

(a) Click the Choose File button.

(b) Select a valid image file (.png or .jpeg).

(c) Submit the upload.

**Expected Results:** The system accepts the image and sends it to the backend for processing. The uploaded image appears in the interface and alt-text generation begins.
**Failure Condition:** The image fails to upload, the UI freezes, or the system does not start alt-text generation.

UII-MT 2. **Initial State:** User is logged in and the main screen is displayed.
**Input:** User attempts to upload an unsupported file type or file exceeding the maximum size.
**Test Steps:**

(a) Click the Choose File button.

(b) Select an unsupported file (e.g., .gif, .pdf) or a file larger than the allowed size.

(c) Submit the upload.

**Expected Results:** The system rejects the file and displays an error message such as "File type not supported" or "File too large."
**Failure Condition:** The system accepts unsupported files or fails to display an error message.

UII-MT 3. **Initial State:** Generated alt text is displayed on the screen.
**Input:** User presses the Copy button to copy the generated alt text.
**Test Steps:**

(a) Generate alt text for an uploaded image.

(b) Click the Copy Alt Text button.

(c) Paste the clipboard contents into another application (e.g., a text editor).

**Expected Results:** The copied text matches the generated alt text exactly.
**Failure Condition:** The clipboard is empty or the copied text is incomplete or incorrect.

UII-MT 4. **Initial State:** Generated alt text is visible and editable on the screen.
**Input:** User edits the alt text and saves the changes.
**Test Steps:**

(a) Modify the alt text by adding, deleting, or changing words.

(b) Click the Save Edits button.

**Expected Results:** The updated alt text is displayed and stored correctly without formatting issues.
**Failure Condition:** The edits are not saved, the text reverts to its original version, or formatting errors occur.

UII-MT 5. **Initial State:** Login screen is displayed.
**Input:** User enters valid credentials and attempts to sign in.
**Test Steps:**

(a) Enter valid login credentials.

(b) Click the Sign In button.

**Expected Results:** The user is authenticated and redirected to the main system interface.
**Failure Condition:** Valid credentials are rejected or no response occurs after clicking Sign In.

UII-MT 6. **Initial State:** User is logged into the system.
**Input:** User presses the Sign Out button.
**Test Steps:**

(a) Click the Sign Out button.

**Expected Results:** The user session ends and the system redirects the user to the login screen.
**Failure Condition:** The user remains logged in or can continue accessing the system without logging in again.

The following table shows the evaluation of the userinterfaceinteractions module's manual tests. Table 13 summarizes the results.

Table 13: UserInterfaceInterfactions Manual Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| UII-MT1 | Upload a valid image file (.png or .jpeg) using the Choose File button | Image is accepted and alt-text generation begins | Image uploaded successfully and alt-text generation initiated | Pass |
| UII-MT2 | Upload an unsupported file type or file exceeding the allowed size | System rejects file and displays an appropriate error message | Unsupported file was rejected and an error message was displayed | Pass |
| UII-MT3 | Copy generated alt text using the Copy Alt Text button | Alt text is copied to clipboard accurately | Alt text copied successfully and matched the generated output when pasted | Pass |
| UII-MT4 | Edit generated alt text and save changes | System saves and displays updated alt text without formatting issues | Edits were saved correctly and updated text was displayed | Pass |
| UII-MT5 | Enter valid credentials and click Sign In | User is authenticated and redirected to the main interface | Login successful and user was redirected to the main screen | Pass |
| UII-MT6 | Click the Sign Out button | User session ends and login screen is displayed | User session ended and system returned to the login screen | Pass |

## 8.2 UserInterfaceAccessibility Module

The following are the manual tests for this module.

UIA-MT 1. **Initial State:** A system screen such as the login screen, main screen, or history screen is loaded.
**Input:** The tester inspects the rendered page structure using browser accessibility tools or screen reader landmarks.
**Test Steps:**

(a) Open a system screen.

(b) Inspect the page structure using accessibility inspector or a screen reader landmark list.

(c) Verify that the main content area is identified as the page's main landmark.

**Expected Results:** The screen contains a valid main landmark (`<main>` or equivalent) for the primary content area.
**Failure Condition:** No main landmark is present, or the primary content is not identified correctly.

UIA-MT 2. **Initial State:** The login screen or another screen (main or history screen )has just loaded.
**Input:** The tester loads the screen and checks where keyboard focus is placed.
**Test Steps:**

(a) Open the login screen.

(b) Do not click the mouse.

(c) Observe which element receives focus automatically.

(d) Press `Tab` to continue keyboard navigation if needed.

**Expected Results:** Keyboard focus is moved to the intended first interactive element, such as the username field on the login screen.
**Failure Condition:** Focus is missing, placed on a non-interactive element, or placed on the wrong control.

UIA-MT 3. **Initial State:** A screen reader is enabled and the user performs an action that changes the UI.
**Input:** Trigger a UI change such as loading the login screen, generating alt text, or navigating to history.
**Test Steps:**

(a) Enable a screen reader such as NVDA, JAWS, or VoiceOver.

(b) Perform an action that changes the interface.

(c) Listen for the announcement made by the system.

**Expected Results:** A clear announcement describing the UI change is read by the screen reader through the live region.
**Failure Condition:** No announcement is made, the message is incomplete, or the announcement is unclear.

UIA-MT 4. **Initial State:** A screen containing multiple interactive elements is open.
**Input:** The tester uses only the keyboard to navigate within the interface.
**Test Steps:**

(a) Open a screen with interactive controls.

(b) Use `tab` to move between elements where applicable.

(c) Use `Enter` or `Space` to activate the selected control.

**Expected Results:** Keyboard navigation works correctly within the defined scope. Focus moves predictably and activation occurs without changing application state unexpectedly.
**Failure Condition:** Tab key do not work, focus movement is inconsistent, or activation fails.

The following table shows the evaluation of the userinterfaceaccessibility module's manual tests. Table 14 summarizes the results.

Table 14: UserInterfaceAccessibility Manual Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| UIA-MT1 | Verify that the loaded screen contains a valid main landmark for primary content | Main content area is identified by a valid main landmark | Main landmark was not present and did not identify the primary content area | Fail |
| UIA-MT2 | Load the screen and check automatic keyboard focus placement | Focus is placed on the intended first interactive element | Initial focus was correctly placed on the expected interactive control | Pass |
| UIA-MT3 | Trigger a UI change while a screen reader is enabled | Screen reader announces the interface change clearly | Announcement was read clearly by the screen reader after any UI update | Pass |
| UIA-MT4 | Use arrow keys, Home/End, and Enter/Space to navigate and activate controls | Keyboard navigation works correctly within the interface scope | Keyboard navigation and activation worked as expected without unexpected state changes | Pass |

**Analysis of Failed Tests:**

- **UIA-MT1:** These failures were due to missing UI elements (e.g., main landmark) and incomplete implementation of accessibility documentation and best practices. The issues stem from implementation gaps and evolving requirements rather than fundamental design flaws. Automated tools helped identify these gaps for future correction.

## 8.3 DataPreProcess Module

The following are the manual tests for this module.

DP-MT 1. **Initial State:** The preprocessing function is available and ready to process dataset examples.
**Input:** A dataset example containing valid image paths and modified alt text.
**Test Steps:**

(a) Provide a dataset example containing valid image paths and alt text.

(b) Run the preprocess function.

(c) Inspect the returned encoded output.

**Expected Results:** The function successfully loads images and text and returns encoded tensors containing image and text representations.
**Failure Condition:** The preprocessing function fails to return encoded outputs or raises an unexpected error.

DP-MT 2. **Initial State:** The preprocessing function is ready to process input images.
**Input:** Image files that may not originally be in RGB format.
**Test Steps:**

(a) Provide image paths to the preprocess function.

(b) Run preprocessing.

(c) Inspect the processed images.

**Expected Results:** All images are converted to RGB format before being passed to the processor.
**Failure Condition:** Images remain in their original format and are not converted to RGB.

DP-MT 3. **Initial State:** The preprocessing function receives alt text label with leading or trailing whitespace.
**Input:** Alt text label containing extra whitespace characters.
**Test Steps:**

(a) Provide alt text label with leading or trailing spaces.

(b) Run preprocessing.

(c) Inspect the processed text values.

**Expected Results:** Leading and trailing whitespace is removed from the alt text.
**Failure Condition:** Whitespace remains in the processed alt text.

DP-MT 4. **Initial State:** The preprocessing function is ready to process dataset examples.
**Input:** A dataset example containing an invalid or unreadable image path.
**Test Steps:**

(a) Provide a dataset example containing an invalid image path.

(b) Run the preprocess function.

(c) Observe the system response.

**Expected Results:** The function raises an error indicating that the image cannot be opened or read.
**Failure Condition:** The function silently proceeds or produces invalid encoded output.
eeds the maximum length.

The following table shows the evaluation of the datapreprocess module's manual tests. Table 15 summarizes the results.

Table 15: DataPreprocess Manual Testing Evaluation

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| DPP-MT1 | Process valid image paths and alt text labels through the preprocess function | Valid images and text are encoded successfully | Images and alt text were processed successfully and encoded tensors were returned | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|
| DPP-MT2 | Provide image paths and run preprocessing to verify RGB conversion | Images are converted to RGB format before encoding | Images were successfully converted to RGB format before processing | Pass |
| DPP-MT3 | Provide alt text containing leading or trailing whitespace | Whitespace is removed from alt text | Alt text whitespace was successfully stripped before encoding | Pass |
| DPP-MT4 | Provide an invalid or unreadable image path to preprocessing | Function raises an error indicating the image cannot be opened | Invalid image path correctly triggered an error during preprocessing | Pass |

## 8.4 ModelOutput Module

Table 16 summarizes the manual testing result for the ModelOutput module.

The following are the manual tests for this module.

MO-MT 1. **Initial State:** The system has successfully generated alt-text for an uploaded image.

**Input:** The backend returns the generated caption to be displayed to the user.

**Test Steps:**

(a) Upload a valid diagram image.

(b) Wait for the alt-text generation process to complete.

(c) Observe the output displayed on the user interface.

**Expected Results:** The generated alt-text is correctly formatted and displayed to the user through the interface.

**Failure Condition:** The generated alt-text is missing, incorrectly formatted, or not displayed to the user.

Table 16: Manual Testing Evaluation for ModelOutput Module

| Test ID | Module | Description | Actual Result | Result |
|---|---|---|---|---|
| MO-MT1 | ModelOutput Module | Verify generated alt-text is correctly formatted and displayed to the user. | All assertions pass | Pass |

## 8.5 AIModelTraining Module

Table 17 summarizes the manual testing result for the AIModelTraining module.

The following are the manual tests for this module.

AI-MT 1. **Initial State:** The training dataset and configuration parameters are available.

**Input:** The model training process is initiated using the dataset.

**Test Steps:**

(a) Start the training pipeline.

(b) Load the dataset used for training the model.

(c) Monitor the training process for completion.

**Expected Results:** The model successfully completes training and produces a trained model artifact used for caption generation.

**Failure Condition:** Training fails to complete, produces errors, or does not generate a usable trained model.

Table 17: Manual Testing Evaluation for AIModelTraining Module

| Test ID | Module | Description | Actual Result | Result |
|---|---|---|---|---|
| AI-MT1 | AIModelTraining Module | Verify that the model training process successfully produces a trained model. | All assertions pass | Pass |

## 8.6 CaptionGeneration Module

Table 18 summarizes the manual testing result for the CaptionGeneration module.

The following are the manual tests for this module.

CG-MT 1. **Initial State:** A trained model exists and a valid image has been uploaded.

**Input:** The system processes the uploaded image using the trained model.

**Test Steps:**

(a) Upload a valid diagram image.
(b) Submit the image for caption generation.
(c) Wait for the caption generation process.

**Expected Results:** The system generates descriptive alt-text that reflects the content of the image.

**Failure Condition:** The system fails to generate alt-text, produces irrelevant text, or returns an error.

Table 18: Manual Testing Evaluation for CaptionGeneration Module

| Test ID | Module | Description | Actual Result | Result |
|---|---|---|---|---|
| CG-MT1 | CaptionGeneration Module | Verify that the system generates descriptive alt-text for uploaded images. | All assertions pass | Pass |

## 8.7 InternalMetricCompliance Module

Table 19 summarizes the manual testing result for the InternalMetricCompliance module.

The following are the manual tests for this module.

IMC-MT 1. **Initial State:** A caption has been generated by the Caption-Generation module.

**Input:** The generated caption is evaluated using internal quality metrics.

**Test Steps:**
(a) Generate alt-text from an uploaded image.
(b) Allow the system to run internal metric validation checks.
(c) Observe whether the caption passes the internal quality checks.

**Expected Results:** The caption is evaluated against internal metrics and confirmed to meet clarity and quality standards.

**Failure Condition:** The caption fails validation checks or the system does not evaluate the generated caption.

Table 19: Manual Testing Evaluation for InternalMetricCompliance Module

| Test ID | Module | Description | Actual Result | Result |
|---|---|---|---|---|
| IMC-MT1 | InternalMetricCompliance Module | Verify that generated captions satisfy internal quality evaluation metrics. | All assertions pass | Pass |

# 9 Changes Due to Testing

During the internal testing phase, several changes were made to the tool based on feedback from the TA, professor, supervisor and from Team 22. A brief description of each change and its implementation is listed below:

## 9.1 Fixing Session Timeout Handling

During testing, it was observed that user sessions did not expire after the defined `SESSION_TIMEOUT` period of inactivity. This posed a potential security risk, as inactive users could remain logged into the system indefinitely. To address this issue, the session management logic was updated to properly track user inactivity and automatically terminate sessions once the timeout threshold was reached. The system now redirects users to the login page and requires re-authentication after inactivity exceeds `SESSION_TIMEOUT`. This change improves system security and ensures compliance with authentication and session management requirements.

## 9.2 Narrowing the Scope

While the team had the ambition to make a STEM-alt text generator, this proved to be very expensive in terms of time needed for training and constraints on available computers for AI model tuning. To make up for this, Team 22 narrowed the scope to only "Physics" based diagrams, with a plan to further narrow the image types to a select list. This change would help our AI model learn better, contextual text and in turn produce higher quality, specific alt text that is usable for several stakeholders like students and teachers.

## 9.3 Shortening Alt-Text Generation Time

The AI model initially took five minutes or more to generate text for one image. After talking to our supervisor, Team 22 was able to decide on a faster model that provided mediocre alt-text generation while adding in indicators for user navigations and processes. We were able to cut down from approximately five minutes per image to 1-2 minutes per image, which significantly reduced user frustration.

## 9.4 Clearer User Navigation

During testing, Team 22 had found that some user navigation processes were not as clear to a screen-reader user or someone with a visual impairment. Examples

include no descriptive 'aria' labels or a misplaced page navigation. To fix this, we tried testing both automatically and manually in several ways including testing the solution with just the keyboard and voice assist. This significantly improved our solution, reduced tester frustration and gave users a clear indication of what was going on behind the scenes.

### 9.5 Future Work

While our testing showed us many faults in our solution, not all of them could be fixed. For future revisions, Team 22 plans to further increase alt-text quality by adding more specificity and context to generation (using AI hypertuning methods). Conducting user testing has given us sufficient data to know what features and system modules to improve for better usability.

## 10 Automated Testing

This section describes the automated testing approaches used to evaluate the system and support overall verification.

### 10.1 Accessibility Checker - Axe

The accessibility checks were done using a third-party website, Axe, which helped Team 22 understand gaps in user navigation for people with disabilities. Figure 1 shows an example of a generated accessibility report.

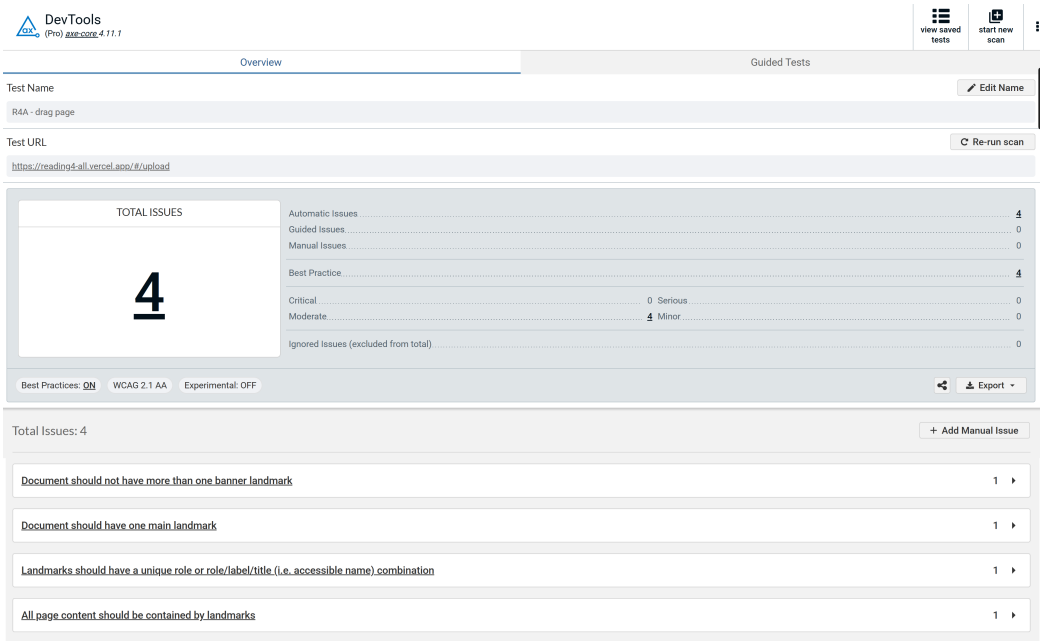


Figure 1: Automated Report for "Upload Page" in the Reading4All System

Reports like these made it easier for the team to identify accessibility oversights and best practices we might have missed and fix them.

## 10.2 CI/CD

Continuous Integration and Continuous Deployment (CI/CD) practices were used to ensure that changes to the Reading4All system could be integrated and tested efficiently. Whenever updates were pushed to the repository, automated workflows were triggered to verify that the system built successfully and that core components continued to function as expected.

These automated checks helped the team detect issues early in the development process and prevented unstable code from being merged into the main branch. By incorporating CI/CD into the development workflow, Team 22 was able to maintain code quality, streamline collaboration, and ensure that new features or fixes did not unintentionally break existing functionality.

Additionally, the CI/CD pipeline supported reproducibility by ensuring that the same build and testing procedures were executed consistently for each update. This contributed to a more stable and reliable development process for the Reading4All system.

## 10.3 Mutation Testing

Mutation testing was originally proposed in the VnV Plan as a method to evaluate the effectiveness of unit tests by introducing small faults (mutations) into the code and verifying whether the existing tests could detect them.

However, mutation testing was not fully implemented in this project due to time constraints and the additional setup complexity required for integrating mutation testing tools with both the React frontend and Python backend environments. Additionally, the team prioritized functional correctness, accessibility testing, and integration validation, which were more critical to the system's goals.

Despite not conducting formal mutation testing, test effectiveness was indirectly assessed through code coverage analysis and the ability of unit tests to detect real defects during development. Future work could include incorporating mutation testing tools to further evaluate and strengthen the robustness of the test suite.

# 11 Trace to Requirements

The following table (Table 20) shows the relationship between the tests performed and the requirements defined in the SRS (2025).

Table 20: Traceability Between Tests and Requirements

| Test ID | SRS Requirement(s) Satisfied |
|---|---|
| IMGVAL-UT1 – IMGVAL-UT5 | FR 1, MD-SL 3, SR-IM 1 |
| BCONT-UT1 – BCONT-UT4 | FR 1, FR 2, FR 4, FR 5 |
| AUTH-UT1 – AUTH-UT6 | FR 6, SR-AR 1, SR-IM 2 |
| MAIN-UT1 – MAIN-UT18 | FR 1, FR 2, FR 4, UHR-EUR 1, UHR-EUR 2 |
| HIST-UT1 – HIST-UT4 | FR 5, UHR-EUR 2 |
| LOGIN-UT1 – LOGIN-UT16 | FR 6, SR-AR 1, SR-PR 1 |
| UII-MT1 – UII-MT6 | FR 1, FR 2, FR 4, FR 6, UHR-PIR 1 |
| UIA-MT1 – UIA-MT4 | LFR-AR 1, LFR-AR 3, LFR-AR 4, UHR-AR 2 |
| DPP-MT1 – DPP-MT4 | PR-SL 1, PR-SL 2, PR-PAR 1 |
| MO-MT1 | FR 2, PR-PAR 2 |
| AI-MT1 | PR-PAR 3 |
| CG-MT1 | FR 2, PR-PAR 1 |
| IMC-MT1 | PR-PAR 1, PR-PAR 2, PR-PAR 3 |
| FR-ST1 – FR-ST7 | FR 1 – FR 6 |
| NFR-ST1 – NFR-ST24 | LFR-AR 1 – LFR-AR 4, UHR-EUR 1 – UHR-EUR 4, PR-SL 1 – PR-SL 2, SR-AR 1, SR-PR 1 |

# 12 Trace to Modules

The following table (Table 21) shows the relationship between the tests performed and the modules defined in the Module Guide. Each test verifies the correct functionality of the corresponding module in the Reading4All system.

Table 21: Traceability Between Tests and Modules

| Test ID | Module |
|---|---|
| IMGVAL-UT1 – IMGVAL-UT5 | ImageValidation Module |
| BCONT-UT1 – BCONT-UT4 | BackendController Module |
| AUTH-UT1 – AUTH-UT6 | AuthenticationService Module |
| MAIN-UT1 – MAIN-UT18 | MainScreen Module |
| HIST-UT1 – HIST-UT4 | ShowHistory Module |
| LOGIN-UT1 – LOGIN-UT16 | LoginScreen Module |
| UII-MT1 – UII-MT6 | UserInterfaceInteractions Module |
| UIA-MT1 – UIA-MT4 | UserInterfaceAccessibility Module |
| DPP-MT1 – DPP-MT4 | DataPreProcess Module |
| MO-MT1 | ModelOutput Module |
| AI-MT1 | AIModelTraining Module |
| CG-MT1 | CaptionGeneration Module |
| IMC-MT1 | InternalMetricCompliance Module |
| FR-ST1 – FR-ST7 | System Functional Tests (cross-module verification) |
| NFR-ST1 – NFR-ST24 | System Nonfunctional Tests (cross-module verification) |

# 13 Code Coverage Metrics

The following analyzes the code coverage metrics for the React frontend of the system. The analysis is based on the coverage data generated using Jest and React Testing Library as shown in Figure 2. Code coverage is a measure of how well the codebase is tested, and it helps identify areas that may require additional testing.

| File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|---|---|---|---|---|---|
| **All files** | **79.27** | **70.52** | **72.05** | **79.93** | |
| src | 0 | 100 | 0 | 0 | |
| App.js | 0 | 100 | 0 | 0 | 12 |
| index.js | 0 | 100 | 100 | 0 | 6-7 |
| src/components | 0 | 0 | 0 | 0 | |
| Header.jsx | 0 | 0 | 0 | 0 | 6-42 |
| src/pages | 100 | 83.33 | 100 | 100 | |
| RequireAuth.jsx | 100 | 83.33 | 100 | 100 | 22 |
| src/pages/HistoryScreen | 87.09 | 80 | 83.33 | 87.09 | |
| HistoryPage.jsx | 87.09 | 80 | 83.33 | 87.09 | 22-26,35,55 |
| src/pages/HomeScreen | 50 | 100 | 33.33 | 50 | |
| homeScreen.jsx | 50 | 100 | 33.33 | 50 | 17-24 |
| src/pages/LogInScreen | 100 | 92.85 | 100 | 100 | |
| login.jsx | 100 | 92.85 | 100 | 100 | 33 |
| src/pages/ShowHistoryScreen | 88.88 | 50 | 80 | 88.88 | |
| showHistory.jsx | 88.88 | 50 | 80 | 88.88 | 38 |
| src/pages/SignUpScreen | 83.33 | 74.28 | 90 | 83.33 | |
| SignUpScreen.jsx | 83.33 | 74.28 | 90 | 83.33 | 42,71,111-112,118,130-131,148-150,156,242 |
| src/pages/UploadScreen | 77.03 | 72.28 | 65.21 | 78.78 | |
| UploadScreen.jsx | 77.03 | 72.28 | 65.21 | 78.78 | 31-32,36,61,65,71-102,117,128-129,134,158,175,192,222-226,321-419 |

**Test Suites:** 7 passed, 7 total
**Tests:** 45 passed, 45 total
**Snapshots:** 0 total
**Time:** 1.543 s, estimated 2 s
Ran all test suites.

Figure 2: Coverage Report of the React Frontend

The following analyzes the code coverage for the backend of the system. This analysis was performed using `pytest-cov` and shows how key features such as authentication, image validation, session history of the system are unit tested. Furthermore, files such as `alt_text_history.py` and `gen_alt_text.py` have lower unit test coverage due to their reliance on Supabase database services. As a result, their functionality was additionally tested manually to ensure correct integration with Supabase and to ensure the expected behavior. Please refer to the Figure 3 for more detail and insight.

```
================================ tests coverage ================================
_______________ coverage: platform darwin, python 3.12.6-final-0 _______________

Name                                          Stmts   Miss  Cover
---------------------------------------------------------------
api/__init__.py                                   0      0   100%
api/admin.py                                      0      0   100%
api/apps.py                                       3      3     0%
api/management/__init__.py                        0      0   100%
api/management/commands/__init__.py               0      0   100%
api/management/commands/test_supabase.py          9      4    56%
api/middleware.py                                22     22     0%
api/migrations/0001_initial.py                    6      6     0%
api/migrations/__init__.py                        0      0   100%
api/models.py                                    14     14     0%
api/tests.py                                      0      0   100%
backend/__init__.py                               0      0   100%
backend/asgi.py                                   4      4     0%
backend/settings.py                              29     29     0%
backend/urls.py                                   3      3     0%
backend/wsgi.py                                   5      5     0%
controller/backend_controller.py                 23      1    96%
databases/connect_supabase.py                    18     10    44%
manage.py                                        11     11     0%
services/alt_text_history.py                     26     10    62%
services/auth_service.py                         16      0   100%
services/edit_alt_text.py                        18      0   100%
services/gen_alt_text.py                         67     19    72%
services/image_validation.py                     18      0   100%
tests/test_alt_text_history.py                   62      0   100%
tests/test_auth_service.py                       71      5    93%
tests/test_backend_controller.py                 31      0   100%
tests/test_edit_alt_text.py                      51      0   100%
tests/test_gen_alt_text.py                       64      0   100%
tests/test_image_validation.py                   43      0   100%
---------------------------------------------------------------
TOTAL                                           614    146    76%
```

Figure 3: Coverage Report of the Backend

# References


Software requirements specification for bridging gaps: Ai for diagram accessibility, 2025. URL https://github.com/4G06-CAPSTONE-2025/Reading4All/blob/main/docs/SRS-Volere/SRS.pdf.

# Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response: None.

**Team Reflection**

1. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

   One aspect that stemmed from speaking with our supervisor is the importance of manual testing over automated testing, specifically the user interface. Our supervisor emphasized the importance of ensuring our interface is accessible and this is highlighted through our various nun-functional tests. We tried our best to put ourselves in the users shoes and understand any difficulties or issues they might face while using the Reading4All application. Furthermore, determining which aspects of the system to unit test verses manually test was determined through discussions with our TA supervisor. This was an aspect we were initialling unsure about; however, after discussions we determined that certain aspects such as the system as a whole, the accessibility, and alt text generation might be better manually tested. The Reading4All system implementation was mainly derived from our previous learnings through courses and industry standards, rather than from our clients and peers.

2. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence

that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

One way that the VnV plan was different from the activities conducted was that we initially planned to test all our modules, prior to us determining and forming our modules. This caused an issue when trying to actually conduct the VnV, as the modules were designed without having the testing in mind. This led us to have modules that were difficult to directly test and have concrete results for. As a result, one modification to the plan and overall system, is that certain modules had to be refactored as they could not be effectively tested and did not have any code we can point to. Furthermore, our VnV plan was dependent on a specific solution, that was vastly modified throughout the year. The VnV plan should have been written in a more generalized manner such that it would be flexible for any solution. For example, earlier in the year our team thought that a feedback module would be an appropriate way to train our AI model, however, through more research and various experiments, we determined that hypertuning would lead to better results. We believe we could not have anticipated this, as based on our AI knowledge at the time, we believed this would lead to the best design and implementation. Looking back, it's important for us to make a testing plan that depends more on the results and functionality that our system will exhibit rather than how it will be implemented as this often changes.

**Fiza's Reflection**

1. What went well while writing this deliverable?

   One aspect that went well while writing this deliverable was the clear structure provided by the VnV template and the previous planning documents. Since the system architecture, modules, and test plans were already defined in earlier deliverables such as the Module Guide and VnV Plan, it was easier to connect the implemented tests to the system components. Collaboration within the team also worked well, as each member was responsible for specific modules and testing sections, which allowed the work to be divided efficiently and integrated into a consistent report format.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   One challenge during this deliverable was ensuring consistency between the different sections of the report, such as aligning manual tests, system tests, and module traceability with the modules defined in the Module Guide. Another difficulty was formatting complex LaTeX tables and ensuring they

compiled correctly while maintaining a consistent style across the document. These issues were resolved through iterative debugging of the LaTeX code and by coordinating with teammates to ensure the testing sections aligned with the previously defined modules and test cases.

**Moly Mikhail's Reflection**

1. What went well while writing this deliverable?

   I think many aspects went well while writing this deliverable. Writing the unit tests for the backend was a smooth process as I was highly familiar with the code base and the different branches to be covered. Another aspect that went well was documenting our unit test results, this also provided another opportunity for me to review the unit tests so far and to determine if any tests were missing and add them.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   One pain point I experienced writing this deliverable is determining the difference between the unit test section and automated testing. At first I was confused on the difference; however, after discussing with our TA, it was clearly they are highly related. Furthermore, it was also difficult determining which aspects of the system should be uni tested, verses manual tested. This was also cleared up through discussions with Chris. We decided the for integration testing it would be more effective to do it manually rather than to implement Selenium.

**Dhruv Sardana's Reflection**

1. What went well while writing this deliverable? I was responsible for writing the frontend modules tests. It helped me understand how to set up tests in Jest and React Testing Library and how to run coverage. Additionally, it helped me identify gaps in the application that were uncovered through testing.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   The challenges I faced were mainly related to setting up the testing environment and writing tests for React components that involve asynchronous operations, such as mocking API calls. I resolved these issues by referring to documentation and examples for Jest and React Testing Library, to ensure

that the tests were correctly structured and covered the necessary functionality. Also another major challenge was to understand how to split up unit tests by modules

**Nawaal Fatima's Reflection**

1. What went well while writing this deliverable? The team was better prepared to work together. We split the doc as per usual and got to work, updating each other as often as possible. Our communication was stronger with each other this time around.

2. What pain points did you experience during this deliverable, and how did you resolve them? My VSCode decided to crash every time I tried to work with it, so I had to ask my teammates to make my edits for me which was really annoying :( The tests we wrote for the VnV Plan didn't accurately encompass the solution we ended up building. They seemed disconnected from the solution itself as some parts had vastly changed from how we thought we'd implement it. This was frustrating because we knew our product worked, we just couldn't show it based on a plan we made before we started coding.

**Casey Francine Bulaclac's Reflection**

1. What went well while writing this deliverable?
   One aspect that went well while writing the Verification and Validation report was being able to directly test our implemented modules and observe the outcomes of the test cases. Running the tests allowed us to clearly see which tests passed and which failed, which helped us better understand the current performance of our tool. This process was valuable because it highlighted areas where the system was functioning correctly as well as areas that could be improved for Revision 1. Another positive aspect was conducting user testing. Gathering feedback from users helped us identify what parts of the tool were working well and which parts needed improvement. Seeing how users interacted with the system provided practical insight and helped validate whether the tool behaved as expected from a user perspective.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   One challenge we encountered was that some modules originally described in the design documentation were not fully implemented in the final system. Because of this, we could not create meaningful verification tests for those modules as originally planned. This required us to revisit the testing plan and modify the VnV report so that the test cases only covered modules that

were actually implemented and testable. To resolve this issue, we updated the relevant documentation to ensure consistency across all project deliverables. This involved adjusting parts of the MIS and MG documents so that they accurately reflected the system that was ultimately implemented. Although this required additional revisions, it ensured that the documentation, design descriptions, and testing procedures were all aligned with the final state of the project.